

Especificación Técnica de la POC: Android Supermarket Scanner

Contexto del Documento: Este documento define la arquitectura, el flujo de datos optimizado y los artefactos de código necesarios para construir la Prueba de Concepto (POC) utilizando una arquitectura Server-side con Firebase Cloud Functions y Gemini (1.5/2.0 Flash) para poblar automáticamente un inventario de supermercado.

1. Arquitectura del Sistema y Flujo de Datos

El diseño prioriza la eficiencia operativa en el dispositivo móvil eliminando la inferencia pesada del cliente. La aplicación requiere conexión activa a internet y utiliza Firebase como orquestador centralizador.

Fase / Paso	Acción en Cliente (Android)	Acción en Servidor / Backend
1. Escaneo inicial	Usa Google ML Kit Barcode Scanning para capturar el código de barras en milisegundos de manera local.	Ninguna (Ejecución local).
2. Validación de existencia	Consulta de forma reactiva en Firebase Firestore si el documento con ID {codigo_barras} ya existe.	Firestore sirve el registro inmediatamente si existe, interrumpiendo el flujo de captura para mostrar la información en pantalla.
3. Captura y Carga	Si no existe, bloquea la UI transitoriamente y toma una o más fotos. Sube los bitmaps estructurados a Firebase Storage bajo la ruta / productos/{codigo_barras}/.	Storage recibe los archivos y actúa como trigger de infraestructura.
4. Inferencia y Estructuración	Muestra un estado en cola (SAVED & QUEUED). Queda escuchando (<i>Listener</i>) el documento correspondiente en Firestore.	Una Cloud Function se dispara ante la carga de imágenes, procesa el lote con Gemini usando responseSchema (JSON) y escribe el resultado directamente en Firestore.

2. Configuración del Entorno de Servidor (Firebase Cloud Functions)

Para asegurar que Gemini retorne exclusivamente datos estructurados y limpios para la base de datos, se define el esquema de salida obligatorio utilizando las capacidades nativas del SDK de Google Gen AI en Node.js.

Definición del Prompt del Sistema (System Instruction)

Eres un sistema automatizado de extracción de datos para inventarios de supermercado. Tu único objetivo es analizar una o más imágenes de un empaque de producto y extraer de forma precisa tres atributos obligatorios: la marca del fabricante, el nombre específico del producto y su gramaje o volumen neto. Debes ignorar textos promocionales, recetas, códigos internos o instrucciones de uso.

Estructura del Esquema de Respuesta (JSON Schema)

```
{
  "type": "object",
  "properties": {
    "marca": {
      "type": "string",
      "description": "La marca principal y reconocible del fabricante (ej: Soprole, Nestlé, Corcolén). Si no es visible, usar 'Genérico'."
    },
    "nombre": {
      "type": "string",
      "description": "Nombre descriptivo del producto comercial, excluyendo la marca (ej: Maní Salado, Leche Entera, Arroz Grado 1)."
    },
    "contenido": {
      "type": "string",
      "description": "La cantidad neta indicada en el empaque incluyendo su unidad de medida (ej: 500g, 1L, 250cc, 12 un)."
    }
  },
  "required": ["marca", "name", "contenido"],
  "additionalProperties": false
}
```

Nota de Deuda Técnica: El uso de `responseSchema` garantiza que la salida de la Cloud Function no requiera validaciones complejas de strings ni limpieza de bloques Markdown (como los delimitadores ````json`). Puede parsearse directamente a un objeto nativo antes de la inserción en la colección de Firestore.

3. Implementación en Cliente (Android Kotlin)

A continuación se detalla la lógica necesaria en el dispositivo Android para el manejo del lector de códigos de barras de ML Kit y la sincronización reactiva con la base de datos distribuida.

Inicialización y Configuración del Escáner Local

```
import com.google.mlkit.vision.barcode.BarcodeScannerOptions
import com.google.mlkit.vision.barcode.BarcodeScanning
import com.google.mlkit.vision.barcode.common.Barcode
```

```
val options = BarcodeScannerOptions.Builder()
    .setBarcodeFormats(Barcode.FORMAT_EAN_13, Barcode.FORMAT_EAN_8, Barcode.FORMAT_UPC_A)
    .build()

val scanner = BarcodeScanning.getClient(options)
```

Lógica de Control de Flujo Híbrido (Firestore-Driven)

```
// Ejecutado una vez que ML Kit detecta el código de barras con éxito
fun procesarCodigoBarras(codigoBarras: String) {
    val db = Firebase.firestore
    val docRef = db.collection("productos").document(codigoBarras)

    docRef.get().addOnSuccessListener { document ->
        if (document != null && document.exists()) {
            // PASO EXTRA: El producto ya existe en la base de datos globales
            mostrarInformacionProducto(
                marca = document.getString("marca") ?: "",
                nombre = document.getString("nombre") ?: "",
                contenido = document.getString("contenido") ?: ""
            )
        } else {
            // El producto es nuevo: se requiere flujo de captura fotográfica
            solicitarCapturaFotografica(codigoBarras)
        }
    }.addOnFailureListener { exception ->
        Log.e("POC_SCANNER", "Error al consultar existencia del código", exception)
    }
}
```

4. Prompt Maestro para Generación Automática (Antigravity)

El siguiente bloque contiene las instrucciones consolidadas listas para ser procesadas por herramientas de generación de código o asistentes de desarrollo de software (Antigravity / Cursor / Copilot).

Genera una aplicación nativa en Android utilizando Kotlin, Jetpack Compose y la arquitectura MVVM, junto con el backend en Firebase. La aplicación debe replicar exactamente el siguiente flujo técnico:

1. Pantalla de Escaneo de Código de Barras:
 - Integra la cámara usando CameraX.
 - Procesa los frames en tiempo real usando Google ML Kit Barcode Scanning (EAN_13, EAN_8).
 - Al detectar un código, realiza una consulta 'get()' a la colección 'productos' de Firebase Firestore utilizando el código de barras como ID de documento (Document ID).
2. Bifurcación Lógica Basada en Datos:
 - CASO A (Existe): Si el documento existe en Firestore, detiene la cámara y despliega un diálogo o pantalla con los campos 'marca', 'nombre' y 'contenido' recuperados de la base de datos.

- CASO B (No Existe): Si el documento no existe, transiciona a una pantalla de captura fotográfica, permitiendo al usuario tomar una o más fotografías del producto.

3. Persistencia y Carga Server-side:

- Sube las imágenes capturadas a Firebase Storage en la ruta `'/productos/{codigo_barras}/{timestamp}.jpg'`.

- Mientras las imágenes se suben, muestra en la UI un estado de 'PROCESANDO EN COLA' (SAVED & QUEUED) no bloqueante utilizando un Snackbar o indicador de progreso discreto.

- Configura un Listener en tiempo real ('addSnapshotListener') sobre el documento '{codigo_barras}' en Firestore. En cuanto los campos pasen de nulos a poblados, actualiza la UI mostrando los datos finales extraídos de forma automática.

4. Backend (Firebase Cloud Function en Node.js/TypeScript):

- Genera una función 'onObjectFinalized' de Firebase Storage que se dispare cuando las imágenes terminen de subirse.

- La función debe agrupar las imágenes del directorio del producto e invocar a la API de Gemini (modelo 'gemini-1.5-flash' o 'gemini-2.0-flash') utilizando SDK de Google Gen AI.

- Pásale al modelo un System Instruction estricto indicando que debe extraer la marca, nombre y contenido/gramaje del producto de supermercado a partir de las fotos.

- Utiliza obligatoriamente la propiedad 'responseSchema' de la API de Gemini configurando un esquema JSON estricto con los campos requeridos: 'marca' (string), 'nombre' (string) y 'contenido' (string).

- El JSON resultante devuelto por Gemini debe ser insertado por la Cloud Function en la colección 'productos' de Firestore, usando el código de barras como ID de documento, completando así el ciclo asíncrono.